Management System Tasks & Implementation Guide

Detailed Functional Specification, Architecture Flows & Annotated Source Code Reference

Restaurant POS & Operations | Backend: Laravel 12/13 API | Frontend: React 19 SPA

Chapa ETB Gateway | Supabase Auth & Realtime | Two-Tier Caching & Keyset Pagination

Table of Contents — Core Management Tasks

Task 1: Order Placement & Multi-Fulfillment
Task 2: Kitchen Queue & Expediting Pipeline
Task 3: Delivery Dispatch & Driver Assignment
Task 4: Cashier Desk & Dual Payment Settlement
Task 5: Floor Tables & Conflict-Free Reservations
Task 6: Dish Catalog & Staff Change Proposals
Task 7: RBAC User Governance & Permissions Matrix
Task 8: Executive Analytics & Operational KPI Reporting

Task 1: Order Placement & Multi-Fulfillment (Dine-In vs Delivery)

`MODULE: ORDERS & POS`

Core Functionality: Allows customers or staff to assemble orders with dynamic line items, select fulfillment mode (`dine_in` with table association or `delivery` with recipient coordinates/phone), select payment method (`cash` or `chapa`), and atomically calculate totals and item snapshot prices inside a database transaction.

1. Backend Code Implementation & Functionality

- `server/app/Http/Controllers/Api/OrderController.php (store: lines 143-260)`

  Executes within `DB::transaction()`. Validates that all menu item IDs are currently available (`is_available: true`). Creates the header `Order` record, inserts `OrderItem` rows snapshotting `unit_price` and `line_total`, and sets order `total`. If Chapa online payment is selected, it dispatches an initialize transaction request to `ChapaPaymentService` and returns the generated `checkout_url`.

- `server/app/Http/Requests/StoreOrderRequest.php`

  Enforces payload validation: `items` array (min: 1), `items.*.menu_item_id` (exists in `menu_items`), `items.*.quantity` (integer, min: 1), conditional requirement for `delivery_address` and `delivery_phone` when `fulfillment_type === 'delivery'`, and table assignment validation for `dine_in`.

- `server/app/Models/Order.php (lines 180-237)`

  Defines status progression and transition guards via `allowedNextStatuses()` and `canUserTransition()`.

```php
// server/app/Http/Controllers/Api/OrderController.php: store() snippet
$order = DB::transaction(function () use ($validated, $user, $fulfillment, $paymentMethod) {
    $menuItemIds = collect($validated['items'])->pluck('menu_item_id')->unique()->all();
    $menuItems = MenuItem::query()->whereIn('id', $menuItemIds)->where('is_available', true)->get()->keyBy

    if ($menuItems->count() !== count($menuItemIds)) {
        throw ValidationException::withMessages(['items' => ['One or more menu items are unavailable.']]);
    }

    $order = Order::query()->create([
        'user_id' => $user->id,
        'table_id' => $fulfillment === Order::FULFILLMENT_DELIVERY ? null : ($validated['table_id'] ?? nul
        'fulfillment_type' => $fulfillment,
        'delivery_address' => $fulfillment === Order::FULFILLMENT_DELIVERY ? trim($validated['delivery_addr
        'delivery_phone' => $fulfillment === Order::FULFILLMENT_DELIVERY ? trim($validated['delivery_phone
        'status' => Order::STATUS_PENDING,
        'payment_method' => $paymentMethod,
        'payment_status' => $paymentMethod === Order::PAYMENT_METHOD_CHAPA ? Order::PAYMENT_PENDING : Orde
    ]);

    $total = 0;
    foreach ($validated['items'] as $line) {
        $item = $menuItems->get($line['menu_item_id']);
        $lineTotal = round((float) $item->price * $line['quantity'], 2);
        OrderItem::query()->create([
            'order_id' => $order->id, 'menu_item_id' => $item->id,
            'quantity' => $line['quantity'], 'unit_price' => $item->price, 'line_total' => $lineTotal,
        ]);
        $total += $lineTotal;
    }
    $order->update(['total' => round($total, 2)]);
    return $order;
});
```

2. Frontend Code Implementation & Functionality

- `client/src/pages/MenuPage.tsx` & `client/src/pages/HomePage.tsx`

Maintains client cart state in local memory. Provides an interactive checkout modal allowing toggle between Dine-In (with table selector) and Delivery (with address and phone inputs), and payment radio toggle (Cash vs Chapa). Invokes `api.ts:createOrder`. If `checkout_url` is returned, automatically redirects customer to the Chapa secure gateway.

- `client/src/pages/OrdersPage.tsx`
  Displays customer's personal order history. Uses `useOrdersRealtime` to automatically update when kitchen staff advance the order. Provides a receipt viewing button navigating to `/orders/:id/receipt`.

Task 2: Kitchen Queue & Live Expediting Pipeline

MODULE: KITCHEN OPERATIONS

Core Functionality: Provides kitchen chefs and expediting staff with an active, real-time ticket display. Filters out unapproved online orders, groups active preparation stages (`pending` , `accepted` , `preparing` , `ready`), and supports one-click status transitions with Supabase Realtime synchronization and 5-second polling fallback.

1. Backend Code Implementation & Functionality

- `server/app/Http/Controllers/Api/OrderController.php (queue: lines 52-89)`

  Requires `can_view_all_orders` authorization. Applies `scopePaymentSettled()` to guarantee only confirmed cash orders or cashier-approved Chapa orders reach the kitchen. Uses cursor pagination (`KeysetPaginator`) ascending by ID for FIFO queue ordering.

- `server/app/Models/Order.php (scopePaymentSettled: lines 307-318)`

  Enforces strict segregation: Cash orders are immediately visible (`payment_status: not_required`), while Chapa orders require both `payment_status: paid` AND `cashier_approved_at IS NOT NULL` .

- `server/app/Http/Controllers/Api/OrderController.php (updateStatus: lines 368-400)`

  Validates transition legality via `$order->canUserTransition($request->user(), $nextStatus)` . Updates order status and invalidates admin dashboard metrics cache.

2. Frontend Code Implementation & Functionality

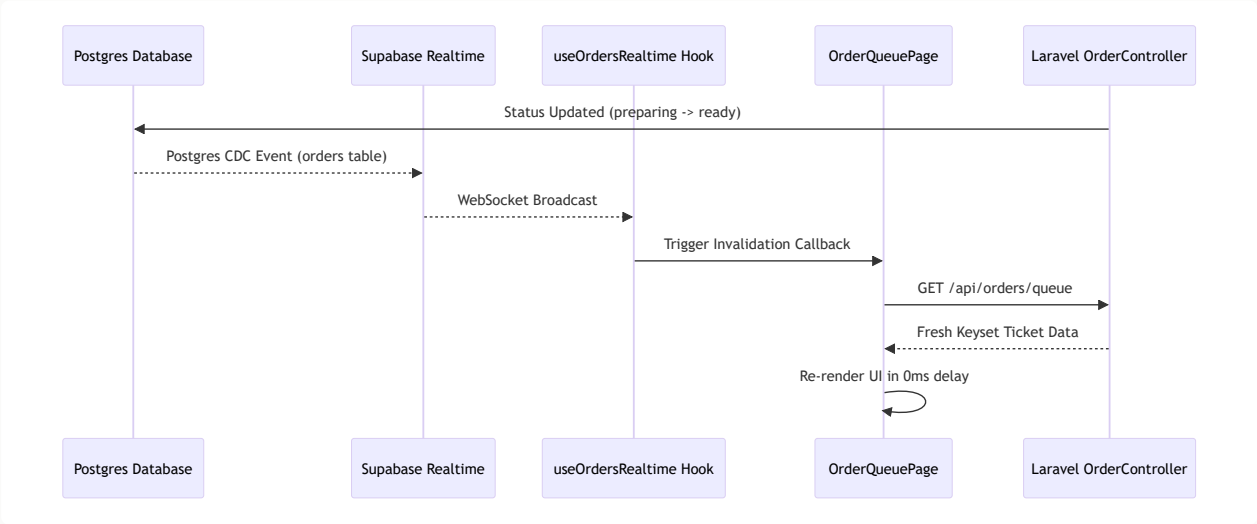
- `client/src/pages/OrderQueuePage.tsx`

  Displays active kitchen tickets organized by status columns with badge indicators, table/delivery tags, customer notes, item quantities, and action buttons ("Start Preparing", "Mark Ready", "Mark Served").

- `client/src/hooks/useOrdersRealtime.ts`

  Subscribes to the Supabase Realtime WebSocket channel on table `orders` . Triggers React Query cache invalidation (`queryClient.invalidateQueries`) immediately upon receiving Postgres change events. Falls back to a 5000ms polling interval if WebSocket disconnects or when running against local SQLite.

FIGURE 1: KITCHEN QUEUE REALTIME & SETTLEMENT FLOW



Task 3: Delivery Dispatch, Driver Assignment & Courier Dual-Confirmation

MODULE: LOGISTICS & DISPATCH

Core Functionality: Dedicated operational hub for delivery orders. Allows dispatchers to accept incoming delivery requests, view available drivers, assign drivers to active orders, track en-route deliveries, and execute dual-confirmation when couriers and customers validate delivery.

1. Backend Code Implementation & Functionality

- `server/app/Http/Controllers/Api/OrderController.php (deliveries: lines 91-117)`
  Requires `can_manage_deliveries` permission. Filters `fulfillment_type === 'delivery'` across active stages (`pending` through `delivered`, `received`).
- `server/app/Http/Controllers/Api/OrderController.php (drivers: lines 119-141)`
  Returns cached list of users possessing `delivery` or `driver` roles via `ApiCacheService::rememberDrivers`.
- `server/app/Http/Controllers/Api/OrderController.php (assignDriver: lines 401-433)`
  Validates order status is eligible for assignment (`accepted`, `preparing`, `ready`, `out_for_delivery`). Ensures the selected user possesses the delivery role before binding `driver_id`.
- `server/app/Models/Order.php (canUserTransition: lines 211-237)`
  Guards that transition to `out_for_delivery` requires a bound driver. Transition to `delivered` can only be triggered by the assigned driver or delivery managers. Transition to `received` can only be triggered by the ordering customer or delivery managers.

2. Frontend Code Implementation & Functionality

- `client/src/pages/DeliveriesPage.tsx`
  Provides tabbed delivery views (`All`, `Active`, `Awaiting Courier`, `In Kitchen`, `Completed`). Includes driver assignment selector dropdown, search query filtering, customer delivery address/phone cards, and courier dual-validation buttons.
- `client/src/services/api.ts (lines 315-334)`
  Exports `fetchDeliveries`, `fetchDeliveryDrivers`, and `assignOrderDriver` connected to TanStack React Query.

Task 4: Cashier Desk Operations & Dual Payment Settlement Gate

`MODULE: CASHIERING & BILLING`

Core Functionality: Centralized payment administration dashboard. Displays Chapa online payments awaiting manual cashier validation (which unlocks kitchen visibility) and uncollected cash orders. Allows cashiers to confirm physical cash receipts, validate Chapa transactions, and generate thermal-styled printable receipts.

1. Backend Code Implementation & Functionality

- `server/app/Http/Controllers/Api/CashierController.php (desk: lines 18-52)`
  Gated by `can_manage_payments` permission. Queries two distinct datasets: (1) `awaiting_chapa_validation`: Orders with `payment_method = 'chapa'`, `payment_status = 'paid'`, and `cashier_approved_at IS NULL`. (2) `open_cash`: Orders with `payment_method = 'cash'`, `cash_collected_at IS NULL`, excluding cancelled orders.
- `server/app/Http/Controllers/Api/CashierController.php (approve: lines 54-99)`
  Cashier approves verified Chapa transaction. Sets `cashier_approved_at = now()` and `cashier_approved_by = $cashier->id`, immediately releasing the ticket to the kitchen queue.
- `server/app/Http/Controllers/Api/CashierController.php (collectCash: lines 101-140)`
  Marks physical currency collected for dine-in or cash-on-delivery orders, stamping `cash_collected_at` and `cash_collected_by`.
- `server/app/Services/ChapaPaymentService.php`
  Handles direct API communication with Chapa Gateway. Initializes ETB transactions, parses return URLs, verifies transaction references (`/v1/transaction/verify/{tx_ref}`), and extracts gateway error messages.
- `server/app/Services/StaffNotificationService.php`
  Dispatches in-app notifications to managers and cashiers upon new cash order placement or Chapa transaction settlement.

2. Frontend Code Implementation & Functionality

- `client/src/pages/PaymentsPage.tsx`
  Two-column cashier desk interface with active count badges. Features interactive action buttons ("Validate Chapa & Send to Kitchen", "Mark Cash Collected") wired with confirmation dialogs (`ConfirmModal`) and audio/visual success indicators.
- `client/src/pages/ReceiptPage.tsx`
  Generates clean, printable customer receipts styled with monospace itemized tables, VAT/tax breakdown, ETB totals, order timestamps, and cashier stamp signatures.

Task 5: Floor Tables & Conflict-Free Table Reservations

`MODULE: FLOOR & RESERVATIONS`

Core Functionality: Comprehensive table inventory and booking system. Manages restaurant floor tables (capacity, name, status: `free`, `reserved`, `occupied`) and guarantees conflict-free table reservations using pessimistic row-level locking and overlapping timeslot assertion algorithms.

1. Backend Code Implementation & Functionality

- `server/app/Http/Controllers/Api/TableController.php`

  Provides public table listing with HTTP ETag caching and `304 Not Modified` negotiation. Offers staff CRUD operations (`store`, `update`, `destroy`) gated by `can_manage_tables`.

- `server/app/Http/Controllers/Api/ReservationController.php (store: lines 55-84)`

  Executes inside a database transaction. Locks the target table row with `RestaurantTable::lockForUpdate()` to prevent race conditions during simultaneous booking attempts.

- `server/app/Services/ReservationAvailabilityService.php`

  Asserts party size does not exceed table capacity and executes overlapping window query: `starts_at < $endsAt AND ends_at > $startsAt AND status = 'confirmed'`. Throws `ValidationException` if conflict detected.

```php
// server/app/Services/ReservationAvailabilityService.php: assertAvailable()
public function assertAvailable(RestaurantTable $table, int $partySize, CarbonInterface $startsAt, CarbonI
{
    if ($table->capacity !== null && $partySize > $table->capacity) {
        throw ValidationException::withMessages(['party_size' => ["Table only seats {$table->capacity} gue
    }

    $conflict = Reservation::query()
        ->where('table_id', $table->id)
        ->where('status', Reservation::STATUS_CONFIRMED)
        ->where('starts_at', '<', $endsAt)
        ->where('ends_at', '>', $startsAt)
        ->exists();

    if ($conflict) {
        throw ValidationException::withMessages(['table_id' => ['Table is already reserved for this time w
    }
}
```

2. Frontend Code Implementation & Functionality

- `client/src/pages/TablesPage.tsx`

  Visual table floor grid showing live status cards. Allows staff to toggle status (Free, Reserved, Occupied) in real-time, adjust guest capacities, and open the instant reservation modal.

- `client/src/pages/ReservationsPage.tsx`

  Reservation log showing booking timeline, start/end times, guest party size, contact notes, and cancellation triggers.

Task 6: Dish Catalog & Staff Change Proposals Governance

`MODULE: MENU MANAGEMENT`

Core Functionality: Two-tiered menu management system. Authorized managers (`can_manage_menu`) can directly create, edit, upload dish images, toggle availability, and delete menu items. Subordinate staff (`can_propose_menu_changes`) can draft item creation or deletion proposals which managers review, approve, or reject with feedback notes.

1. Backend Code Implementation & Functionality

- `server/app/Http/Controllers/Api/MenuController.php`
  `index()` : Public menu browse cached with ETags and `Cache-Control: stale-while-revalidate` .
  `manage()` : Staff catalog view returning all available and unavailable items.
- `server/app/Http/Controllers/Api/MenuItemController.php`
  Handles direct CRUD operations and image uploads to storage disk via `uploadImage()` . Automatically purges public and staff menu caches on mutation.
- `server/app/Http/Controllers/Api/MenuChangeRequestController.php`
  Endpoints for staff proposal submission (`store`), list retrieval (`index`), manager approval (`approve`), and manager rejection (`reject`).
- `server/app/Services/MenuChangeRequestService.php`
  Applies approved proposals: creates new `MenuItem` from payload or deletes targeted `MenuItem` , stamps reviewer metadata, and flushes menu and dashboard RAM caches.

2. Frontend Code Implementation & Functionality

- `client/src/pages/ManageMenuPage.tsx`
  Multi-tab management console featuring: (1) Direct Edit Tab with category accordions and image uploader, (2) Propose Changes Tab for staff, and (3) Review Queue Tab displaying pending proposal cards with Approve/Reject action triggers.

Task 7: RBAC User Governance, Role Assignment & Permission Matrix

`MODULE: SECURITY & ADMIN`

Core Functionality: Comprehensive Role-Based Access Control (RBAC) governance. Allows administrators to provision new user accounts (synced with Supabase Auth), assign roles (`admin`, `manager`, `cashier`, `kitchen`, `host`, `delivery`, `customer`), and customize permissions assigned to each role through a live matrix grid.

1. Backend Code Implementation & Functionality

- `server/app/Http/Controllers/Api/UserPermissionController.php`
  Gated by `can_manage_permissions`.
  `index()`: Keyset-paginated list of users with assigned roles and computed permissions.
  `store()`: Provisions user account in Supabase via `SupabaseAdminUserService`, inserts local `users` record, and syncs initial roles.
  `updateUserRoles()`: Updates user's role assignments and clears their auth cache.
  `updateRolePermissions()`: Syncs permission IDs for a specific role and flushes auth caches for all users holding that role.
- `server/app/Models/User.php (authProfile: lines 57-82)`
  Calculates user's effective permissions by flattening all assigned roles' permissions and always adding `can_place_order`. Result is cached in RAM and Redis/File store via `ApiCacheService::rememberUserAuth`.
- `server/app/Providers/AppServiceProvider.php (boot: lines 57-65)`
  Registers Laravel authorization Gates dynamically for every permission in `Permission::catalog()`.

2. Frontend Code Implementation & Functionality

- `client/src/pages/AdminRolesPage.tsx`
  Two-panel administrative interface: (1) User Account Management (create user modal, role assignment dropdowns, account deletion), and (2) Role-Permission Matrix (interactive checkbox grid to toggle granular permissions for any role in real-time).
- `client/src/store/AuthContext.tsx (hasPermission: lines 153-156)`
  Provides client-side permission evaluation (`hasPermission(PERMISSIONS.CAN_MANAGE_PAYMENTS)`) to conditionally render UI action buttons and protect client routes.

Task 8: Executive Analytics, Revenue Auditing & Operational KPIs

`MODULE: ANALYTICS & AUDIT`

Core Functionality: Executive business intelligence dashboard aggregating restaurant metrics across customizable date ranges. Computes total sales, paid revenue in ETB, average order value (AOV), top 8 best-selling dishes with revenue attribution, delivery dispatch health, floor table occupancy rates, and pending cashier queue totals.

1. Backend Code Implementation & Functionality

- `server/app/Http/Controllers/Api/AdminDashboardController.php`

  Accepts `from` and `to` ISO timestamp query parameters (defaults to trailing 7 days). Gated by `can_manage_permissions`. Caches aggregated results in `ApiCacheService::rememberAdminDashboard` using SHA256 hashed date-range keys.

- `Calculated Analytics Aggregations:`
  - **Orders & Revenue:** Total volume, status distribution, paid order count, paid revenue sum, average order value (AOV).
  - **Top Selling Dishes:** Joins `order_items` with `orders`, aggregates `quantity_sold` and `revenue`, filters out cancelled orders, and retrieves top 8 items.
  - **Payment & Cashier Health:** Number of Chapa orders awaiting cashier validation, uncollected cash order count.
  - **Logistics:** Active deliveries, unassigned delivery orders, completed deliveries.
  - **Reservations:** Confirmed, cancelled, and upcoming bookings count.
  - **Floor Occupancy:** Instant breakdown of tables by status (`free` , `reserved` , `occupied`).

2. Frontend Code Implementation & Functionality

- `client/src/pages/AdminDashboardPage.tsx`

  Comprehensive executive dashboard with custom date-range picker presets (Today, 7 Days, 30 Days, Custom). Displays KPI summary cards, interactive revenue graphs, dish ranking tables, and operational alert counters with automatic prefetching via `api.ts:prefetchAdminDashboard`.